

COMP0222 / COMP0249 Lab 06: Running and Evaluating ORB-SLAM2

24th February, 2026

1 Scope and Purpose

In this lab, you'll gain some experience working with ORB-SLAM2 using the standard datasets. You'll also work with the evo evaluation tool to evaluate ORB-SLAM2 with respect to the ground truth.

We'll also show how to use COLMAP with evo to evaluate the performance of ORB-SLAM2 on a custom dataset.

2 Activities

Activity 1: Build and Install ORB-SLAM2

ORB-SLAM2 is a C++ native application which you will need to download and build from source.

The source code and build instructions can be found at https://github.com/UCL/COMP0222-249_25-26_ORB_SLAM2. This is a fork and modification of ORB-SLAM2 to make it easier to build and use.

Activity 2: Run on the FR1/xyz TUM Image

We are going to first run on the `fr1/xyz` dataset. This dataset has been shown in the lectures — it consists of a camera largely translating over a desk scene with a monitor. It's one of the simplest datasets, but shows interesting results.

First run `mono_tum` and `rgbd_tum` on the dataset. (You will need to experiment a bit to get the command line correct — check the github repo and grab a TA if you still aren't winning.) We will be using the stored trajectories later, so you should save them as, for example, `fr1xyz-mono.txt` and `fr1xyz-rgbd.txt`. What do you notice about the behaviour of ORB-SLAM in each case? Which is more robust?

We are now going to see how changing the number of ORB features changes the behaviour of the system. Copy the file `TUM1.yaml` from the install directory (see `Install/etc/orbslam2/Monocular`) to

the current directory. Edit this file and change the value assigned to `ORBextractor.nFeatures`. Try values which are larger and smaller. Does this make a difference?

Activity 3: Install the evo Toolkit

Now that you are able to run ORB-SLAM, we'll now install the evo toolkit. evo is a Python-based toolset for evaluating the performance of SLAM systems using ATE (APE), RTE, and provides support for alignment and scaling.

The source code and installation instructions can be found at <https://github.com/MichaelGrupp/evo>. Follow these to download and install the toolkit. You will need to create a virtual environment of some kind to support it.

I believe that it is okay to run the compiled ORB-SLAM from the activated environment, so you can run everything in the same terminal.

Activity 4: Compare the Mono and RGB-D Results for FR1/xyz

We are now going to use evo to compare the mono and RGB-D trajectories you derived earlier.

Run the command:

```
evo_traj tum fr1xyz-mono.txt --ref=fr1xyz-rgbd.txt -p
```

This command tells evo to plot trajectories stores in the `tum` format. The `ref` specification tells evo that the RGB-D dataset is considered the ground truth one. The `-p` argument tells it to plot the results on the screen. The plot itself consists of several tabs, including ones which will show position and orientation errors.

By default this generates a 3D plot. To see a 2D plot of the `xz` values, use the command:

```
evo_traj tum fr1xyz-mono.txt --ref=fr1xyz-rgbd.txt -p --plot_mode xz
```

What results do you get? Why is alignment important?

Compare the results with the alignment using the command line argument `-as`:

```
evo_traj tum --ref=fr1xyz-rgbd.txt fr1xyz-mono.txt -p -as
```

APE and RPE can be computed using the below commands. Note that the `--ref` option should not be used:

```
evo_ape tum fr1xyz-rgbd.txt fr1xyz-mono.txt -p -as
evo_rpe tum fr1xyz-rgbd.txt fr1xyz-mono.txt -p -as
```

Activity 5: Run EuRoC

We are now going to test the EuRoC dataset. This is a mono / stereo dataset which, unlike TUM, has ground truth. We have checked with the Machine Hall 1 (MH01) dataset, but similar instructions should apply to all the others. Down below, it's assumed that MH01 is being used and has been uncompressed to the directory `EuRoC/MH_01`.

Download and uncompress the EuRoC dataset. You should be able to visualize the EuRoC ground truth positions (from a surveying instrument called a total station) using the command:

```
evo_traj euroc EuRoC/MH_01/mav0/state_groundtruth_estimate0/data.csv --plot
```

Run mono EuRoC in mono and stereo mode (check github to see what the commands mean):

```
mono_euroc EuRoC.yaml EuRoC/MH_01/mav0/cam0/data MH01.txt mh01-mono.txt
stereo_euroc EuRoC.yaml EuRoC/MH_01/mav0/cam0/data \
EuRoC/MH_01/mav0/cam1/data MH01.txt mh01-stereo.txt
```

You can now use `evo` to compare both sets with the ground truth:

```
evo_traj euroc mh01-mono.txt mh01-stereo.txt \
--ref=EuRoC/MH_01/mav0/state_groundtruth_estimate0/data.csv -p
```

Activity 6: Run KITTI

We're going to finish up the pre-canned data by testing with KITTI. This is the urban environment dataset which we've demonstrated. As indicated on the github page, the download is very large because it contains 20 different runs within it. We will just be using the KITTI-00 dataset. We have downloaded and extracted this separately. It can be obtained from:

https://drive.google.com/file/d/177g-30g2H3D_g3Bi4ElNV8t-420rezNx/view?usp=sharing

You can run both the mono and stereo versions.

We will be updating these instructions to get the KITTI / TUM conversion working. This is necessary because mono will only be written out in TUM format and so some data conversions are required.

Activity 7: Run the UCL/Kenilworth-3 Dataset

In this activity, we are going to use the toolchain which was used to produce the Kenilworth-3 Dataset which has been shown in the lectures. This was collected using an iPhone. There is no ground truth.

We convert the recorded data to the TUM format, since this is the easiest to work with.

Using the code consists of two main stages:

1. Extrating the frames and creating the timestamp file.
2. Use COLMAP to calibrate the camera

1. Download the Dataset

The dataset can be downloaded from:

https://liveuclac-my.sharepoint.com/:f:/g/personal/ucacsjj_ucl_ac_uk/IgBQke4GC-8XQIpYcpM4j8hWAdyaaS4_rYaLRudWZhTtWtk?e=dB3bZa

It consists of a video file (Kenilworth-3/A001_09131347_C005.mov) as well as teh files:

- `extract_frames_and_build_timestamp_file.sh` which extracts the frames and timestamp files from the video
- `run_colmap.sh` which runs a GUI-free version of COLMAP to calibrate the camera
- `camera_intrinsics.yaml` the calibration parameters we worked out, but you should try them yourselves.

2. Extract the Frames and Create the Timestamp File

In the first step, run the script:

```
./extract_frames_and_build_timestamp_file.sh Kenilworth-3/A001_09131347_C005.mov
```

This uses `ffmpeg` (which you might have to instal it!) to extract each frame of the video and putt it in a directory called `Kenilworth-3/rgb`. Images are downsampled to be 960 pixels wide (which ORB-SLAM likes). The file names are based directly on the timestamp of each individual frame. A file called `Kenilworth-3/rgb.txt` will also be created. This file lists, for each extracted frame, the timestamp obtained.

3. Run ORB-SLAM

You can now run ORB-SLAM directly on this data using the precomputed configuration file:

```
mono_tum Kenilworth-3/camera_intrinsics.yaml Kenilworth-3 \
k3-mono.txt
```

As you've seen, this will take quite some time to run but should illustrate multiple loop closure events.

4. Calibrate the Camera

COLMAP can be used to calibrate the camera. You can run it on the whole dataset (this will take a long time), or you can copy a subset of frames over.

Create a subdirectory `Kenilworth-3/colmap_frames` and copy into it a subset of frames. From the `rgb` directory. I'd recommend looking at the first 100 or so frames. These are of the camera looking at a planar wall. You should pick frames where the camera has translated. You should then be able to run:

```
./run_colmap_sh Kenilworth-3
```

This will run through the whole of COLMAP. Things to check are:

1. COLMAP is able to initialize. You'll see a lot of log messages of the form "trying to triangulate". You should eventually see that it's successful, and it starts to say "adding frames" and "global bundle adjustment".
2. COLMAP can complete sparse reconstructions. In the subdirectory `colmap_results`, a `sparse` subdirectory is created. Within it all the sparse models are stored. If things are good, COLMAP will create a single model in the directory 0. If it has problems, it will generate multiple sparse models in different numbered directories.

Once this has finished, run `colmap gui` and import the models from the `sparse` directory. If there are multiple models, you want to pick the one with the largest number of frames. You'll find within the sparse model directory the file `camera_intrinsics.yaml` which you can use with ORB-SLAM.

If COLMAP fails, you'll have to add more images. At the moment, the script is a bit dumb — you'll have to delete the `colmap_output` directory and run it again.